

The Helm Chart

Packaging and deploying ipecho to Kubernetes with configurable, hardened defaults

Carlos Pereyra
July 20, 2026

The Helm chart packages the ipecho service for Kubernetes. It exposes probes, resources, security context and geo-IP behaviour through values.yaml, ships a hardened security posture by default, and provisions the GeoLite2 database at runtime with an init container - without ever baking a secret into the image or the chart.

FIELD	VALUE
Chart	charts/ipecho
API version	v2 (Helm 3)
Kubernetes objects	Deployment, Service, Ingress, ServiceAccount
Validation	helm lint (runs in CI)
Deploy target	K3s (Traefik ingress); kind in CI

1. Chart layout

```
charts/ipecho/  
  Chart.yaml  
  values.yaml  
  .helmignore  
  templates/  
    _helpers.tpl  
    deployment.yaml  
    service.yaml  
    ingress.yaml  
    serviceaccount.yaml
```

2. Configurable values

Everything an operator typically tunes is exposed in values.yaml: replica count, image, service, ingress, resource requests/limits, the three probe types, the security context, and the geo-IP init container. Sensible, safe defaults are provided.

```
replicaCount: 1

image:
  repository: sauay/ipecho
  tag: ""
  pullPolicy: IfNotPresent

service:
  type: ClusterIP
  port: 80

resources:
  requests: { cpu: 25m, memory: 32Mi }
  limits:   { cpu: 100m, memory: 128Mi }
```

Configurable probes

Liveness, readiness and (optional) startup probes are all driven from values, so operators can tune timings or disable a probe without editing templates.

```
probes:
  liveness:
    enabled: true
    path: /health
    initialDelaySeconds: 5
    periodSeconds: 10
  readiness:
    enabled: true
    path: /ready
  startup:
    enabled: false
```

3. Security hardening

The chart applies a restrictive security posture by default: the container runs as a non-root user, with a read-only root filesystem, all Linux capabilities dropped, no privilege escalation, and the RuntimeDefault seccomp profile. Because the root filesystem is read-only, an empty-Dir is mounted at /tmp.

```
podSecurityContext:
  runAsNonRoot: true
  runAsUser: 1000
  fsGroup: 1000
  seccompProfile: { type: RuntimeDefault }

securityContext:
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
  capabilities:
    drop: [ALL]
```

4. The GeoLite2 init container

The application needs the GeoLite2-City database, which is licensed and too large to bake into the image. The chart solves this with an init container: on pod start it downloads the current database from MaxMind using a license key read from a Kubernetes Secret, extracts it into a shared emptyDir volume, and the application container reads it from there. The database is therefore never committed to Git and never stored in the image, and it refreshes on every pod start.

```
initContainers:
- name: geoip-download
  image: alpine:3.20
  command: ["/bin/sh", "-c"]
  args:
  - |
    wget -O /tmp/db.tar.gz \
      "https://download.maxmind.com/app/geoip_download?edition_id=GeoLite2-
City&license_key=$LICENSE_KEY&suffix=tar.gz"
    tar -xzf /tmp/db.tar.gz -C /tmp
    cp /tmp/GeoLite2-City_*/GeoLite2-City.mmdb /data/GeoLite2-City.mmdb
  env:
  - name: LICENSE_KEY
    valueFrom:
      secretKeyRef:
        name: ipecho-geoip
        key: license
  volumeMounts:
  - name: geoip
    mountPath: /data
```

The main container mounts the same volume read-only and receives GEOIP_DB pointing at the extracted file. The whole feature is gated by geoip.enabled, so the chart still installs (and helm lint / CI kind deploy still pass) with geo-IP turned off.

5. Secrets handling

No secret is hardcoded anywhere in the chart. The only secret the chart consumes is the MaxMind license key, provided as a Kubernetes Secret and referenced with secretKeyRef. The README documents the supported options for application secrets in general: plain env for non-sensitive config, a Kubernetes Secret with envFrom/valueFrom, sealed-secrets to keep an encrypted secret in Git, or the External Secrets Operator backed by a vault.

SECRET	PROVIDED AS	CONSUMED BY
ipecho-geoip	Kubernetes Secret (key: license)	init container (MaxMind download)
Docker Hub token	GitHub repository secret	CI push step (not the chart)

6. Deploying

The chart is installed with a K3s-specific values overlay that enables the ingress (devops-lab.pereyra.ar) and the geo-IP init container. The Service is ClusterIP; Traefik routes the public host to it.

```
# create namespace + geoip secret, then install/upgrade
kubectl apply -f deploy/namespace.yaml
kubectl -n devops-lab create secret generic ipecho-geoip \
  --from-literal=license=$MAXMIND_LICENSE_KEY

helm upgrade --install ipecho charts/ipecho -n devops-lab \
  -f deploy/values-k3s.yaml --set image.tag=<sha> --wait
```

7. A fix worth remembering

During the first cluster deploy the pods never scheduled. The ReplicaSet reported: serviceaccount "ipecho" not found. The chart set serviceAccount.create: true and referenced the account, but the ServiceAccount template itself was missing. Adding templates/serviceaccount.yaml (guarded by serviceAccount.create) fixed it. It is a good example of why the CI kind deploy matters - it would have caught the same error before production.

The chart is fully parameterised, hardened by default, provisions its geo database at runtime from a Secret with zero hardcoded credentials, and is validated by helm lint and an ephemeral kind deployment on every CI run.